



First steps to R for Humanities & Social Sciences

About

Who is this course for?

This course aims to provide a starting point for students and researchers from the Humanities and Social Sciences interested in learning R. This course is for beginners. After completing the four modules, you can continue your learning by participating in other open online courses, joining a Carpentries workshop, working through available open educational resources, or joining a local community of practice.

This introductory course is offered as part of the [ESCALATOR programme](#), an initiative of the [South African Centre for Digital Language Resources](#).

Workshop format

Instructors will work through software installation and the very first steps of using R with open and free training resources. This is a hands-on code-along course, meaning candidates will work on their computers and follow along with the instructor as they work through the material.

At the end of this course, you will be able to:

- Recognise terminology used in programming
- Open the programming environment on your computer
- Run some basic code
- Load and inspect tabular data
- Know where to find help to continue on the journey
- Identify next steps

Funding

These lessons form part of the ESCALATOR programme developed by the South African Centre for Digital Language Resources (SADiLaR). SADiLaR is a national centre supported by the Department of Science and Innovation (DSI).

Data

The data used in this course is the NCHLT English Text Corpora that you can download from the [SADiLaR Resource Catalogue](#).

This data is a collection consisting of a clean corpus, lexicon, frequency list and named-entity lists developed during the *NCHLT Text* project.

Lesson contributors

- Anelda van der Walt
- Bianca Peterson
- Shani Le Roux
- Nomalungelo Maphanga

License

This course is licenced as CC-BY 4.0.

The data is licensed as [Creative Commons Attribution 2.5 South Africa License](#).

Programming software installations

Install R and RStudio

R and RStudio are two separate pieces of software:

- R is a programming language that can be used for computation and number-crunching. It is especially powerful for data exploration, visualization, and statistical analysis
- RStudio is the front-end, also called an integrated development environment (IDE), where we can create and run R code. In this course we use RStudio to interact with R.

If you don't already have R and RStudio installed, follow the instructions for your operating system below. You have to install R before you install RStudio.

Windows

- Download R from the [CRAN website](#).
- Run the **.exe** file that was just downloaded.
- Go to the [RStudio download page](#).

- Under *All Installers*, download the RStudio Installer for Windows.
- Double click the file to install it.
- Once it's installed, open RStudio to make sure it works and you don't get any error messages.

MacOS

- Download R from the [CRAN website](#).
- Select the `.pkg` file for the latest R version.
- Double click on the downloaded file to install R.
- It is also a good idea to install [XQuartz](#) (needed by some packages).
- Go to the [RStudio download page](#).
- Under *All Installers*, download the RStudio Installer for MacOS.
- Double click the file to install RStudio.
- Once it's installed, open RStudio to make sure it works and you don't get any error messages.

Linux

- Follow the instructions for your distribution from [CRAN](#), they provide information to get the most recent version of R for common distributions. For most distributions, you could use your package manager (e.g., for Debian/Ubuntu run `sudo apt-get install r-base`, and for Fedora `sudo yum install R`), but we don't recommend this approach as the versions provided by this are usually out of date. In any case, make sure you have at least R 3.3.1.
- Go to the [RStudio download page](#).
- Under *All Installers*, select the version that matches your distribution and install it with your preferred method (e.g., with Debian/Ubuntu `sudo dpkg -i rstudio-YYYY.MM.X-ZZZ-amd64.deb` at the terminal).
- Once it's installed, open RStudio to make sure it works and you don't get any error messages.

Update R and RStudio

If you already have R and RStudio installed, first check if your R version is up to date:

- When you open RStudio your R version will be printed in the console on the bottom left. Alternatively, you can type `sessionInfo()` into the console. If your R version is 4.0.0 or later, you don't need to update R for this lesson. If your version of R is older than that, download and install the latest version of R from the R project website [for Windows](#), [for MacOS](#), or [for Linux](#)

- It is not necessary to remove old versions of R from your system, but if you wish to do so you can check [How do I uninstall R?](#)
- Note: The changes introduced by new R versions are usually backwards-compatible. That is, your old code should still work after updating your R version. However, if breaking changes happen, it is useful to know that you can have multiple versions of R installed in parallel and that you can switch between them in RStudio by going to **Tools > Global Options > General > Basic**.
- After installing a new version of R, you will have to reinstall all your packages with the new version. For Windows, there is a package called **installr** that can help you with upgrading your R version and migrate your package library.

To update RStudio to the latest version, open RStudio and click on **Help > Check for Updates**. If a new version is available, follow the instructions on the screen to update RStudio. By default, RStudio will also automatically notify you of new versions every once in a while.

*** The installation instructions were copied from the Data Carpentry lesson materials at <https://datacarpentry.org/R-ecology-lesson/#software-setup>*

Zoom installation

Please ensure you have the latest version of Zoom installed to get the best out of our virtual meetups.

Download Zoom from <https://zoom.us/download>.

If you are experiencing any challenges, please send an email to first-steps-program@talarify.co.za.

Weekly Zoom session

Join Zoom Meeting

To join the weekly Zoom training sessions, please click on the link, or use the meeting details below.

<https://us06web.zoom.us/j/4865132416?pwd=ZTVWQWpJWi9LdXhVQUM2SDh1d0NnZz09>

Meeting ID: 486 513 2416

Passcode: 870923

Week 1: Getting started

Our learning goals for this week

1. Understand tidy data
2. Understand the different elements in the programming environment
3. Open the relevant software to use on your computer
4. Describe how programming can be applied in specific contexts in humanities or social sciences
5. Be familiar with the user interface
6. Set up a computational project according to best practices

Understand tidy data

What does tidy data look like?

1. Put all your variables in columns - the thing you're measuring, like 'weight' or 'temperature'.
2. Put each observation in its own row. An observation is a single entity (person, animal, etc.).
3. Don't combine multiple pieces of information in one cell. Sometimes it just seems like one thing, but if you can split the data into two separate columns, then do that instead!
4. Leave the raw data raw - don't change it!
5. Export the cleaned data to a text-based format like CSV (comma-separated values) format. This ensures that anyone can use the data, and is required by most data repositories.

Example of untidy data

Date collected	Plot	Species-Sex	Weight
1/9/78	1	DM-M	40
1/9/78	1	DM-F	36
1/9/78	1	DS-F	135
1/20/78	1	DM-F	39
1/20/78	2	DM-M	43
1/20/78	2	DS-F	144
3/13/78	2	DM-F	51
3/13/78	2	DM-F	44
3/13/78	2	DS-F	146

Example of tidy data

Date collected	Plot	Species	Sex	Weight
1/9/78	1	DM	M	40
1/9/78	1	DM	F	36
1/9/78	1	DS	F	135
1/20/78	1	DM	F	39
1/20/78	2	DM	M	43
1/20/78	2	DS	F	144
3/13/78	2	DM	F	51
3/13/78	2	DM	F	44
3/13/78	2	DS	F	146

Elements in the programming environment

Basic programming elements

The basic elements in programming include:

- input,

- objects (also called variables),
- functions,
- output, and
- data structures.

Input

When you work in RStudio, you can provide different data structures as input. This can range from a single value, to an entire table with data. You can even use images as input!

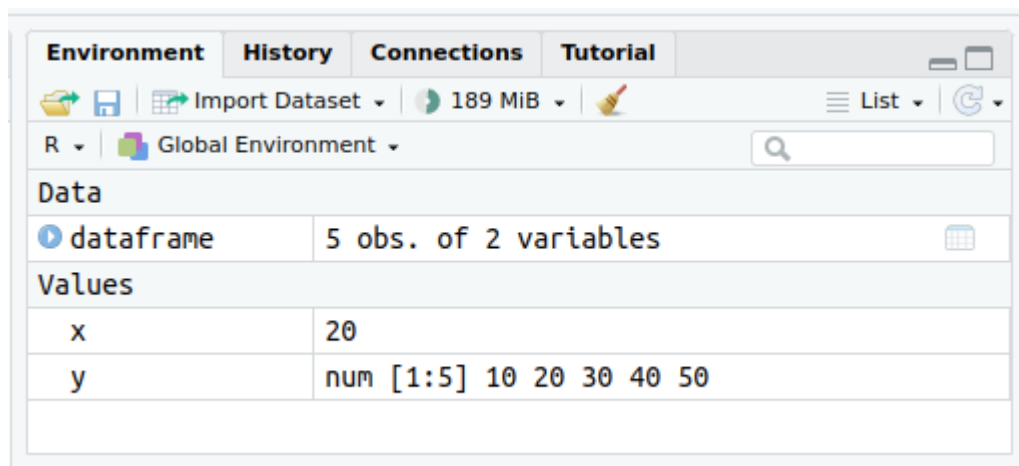
 Examples of input:

- A single value: $x = 25$
- A numeric vector with multiple values: $y = c(10, 20, 30, 40, 50)$
- A table (also called a dataframe) with multiple columns and rows

Objects

When you create or import the data into RStudio, it is saved as an object in R's working memory, called the Environment. We'll use these objects to do calculations and make visualisations.

 Examples of objects in the Environment:



Functions

A function is a set of instructions that you run on the input (i.e., the data) to return the desired output. For example, the `sum()` function will calculate the sum of multiple values, whereas the `min()` and `max()` functions will return the minimum and maximum value, respectively.



Examples of functions:

- `sum(10, 20, 30, 40, 50)` -> will return the value 150
- `min(10, 20, 30, 40, 50)` -> will return the value 10
- `max(10, 20, 30, 40, 50)` -> will return the value 50

Output

If you do calculations or you transform the data (input), the output will either be:

- displayed in the R console if you did not assign it to an object;
- saved as an object in the Environment if you used the assignment operator;
- displayed in the Viewer pane if it's a visualisation; or
- written to an output file (e.g. save the dataframe as a CSV or Excel file, or save the visualisation as a PNG file).

Open the relevant software on your computer

Open RStudio

To open RStudio on your computer, follow the steps below for your operating system.

Windows

1. Click on the  Windows symbol and type *RStudio* in the search bar.
2. Click the  RStudio icon.

MacOS

1. Click on the  Search icon and type *RStudio* in the search bar.
2. Double-click the  RStudio icon.

Linux

1. Click the  Show Applications icon and type *RStudio* in the search bar.
2. Click the  RStudio icon.

Applications of programming

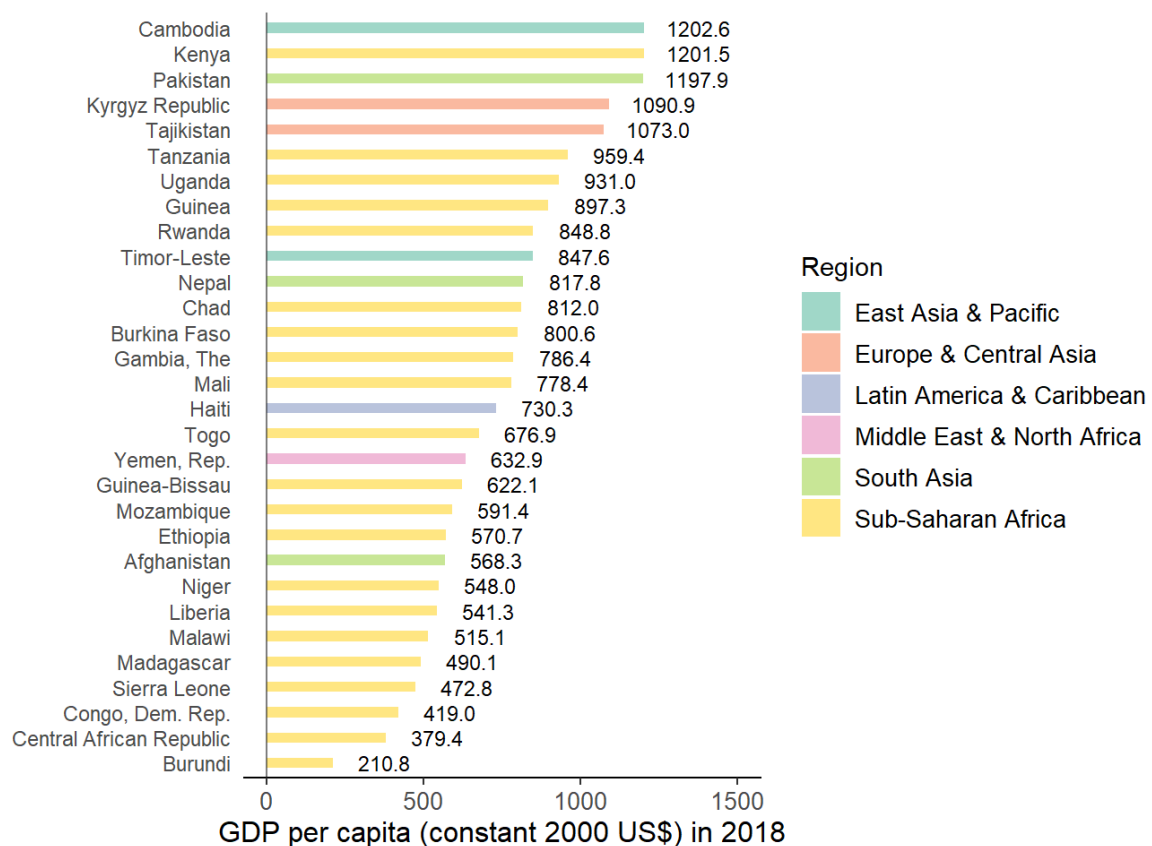
Applications of R programming

The R programming language is widely used in social sciences and humanities for various data analysis, visualization, and research purposes. Here are some real-world examples of its applications:

Economic Analysis

Researchers use R to analyse economic data, estimate econometric models, and conduct policy evaluations. For example, R can be used to analyse labour market trends, income inequality, and the impact of government policies on economic outcomes.

Visit the [R Econ Visual Library](#) for R code for data visualisations in economics, such as the figure below, created and maintained by DIME Analytics.

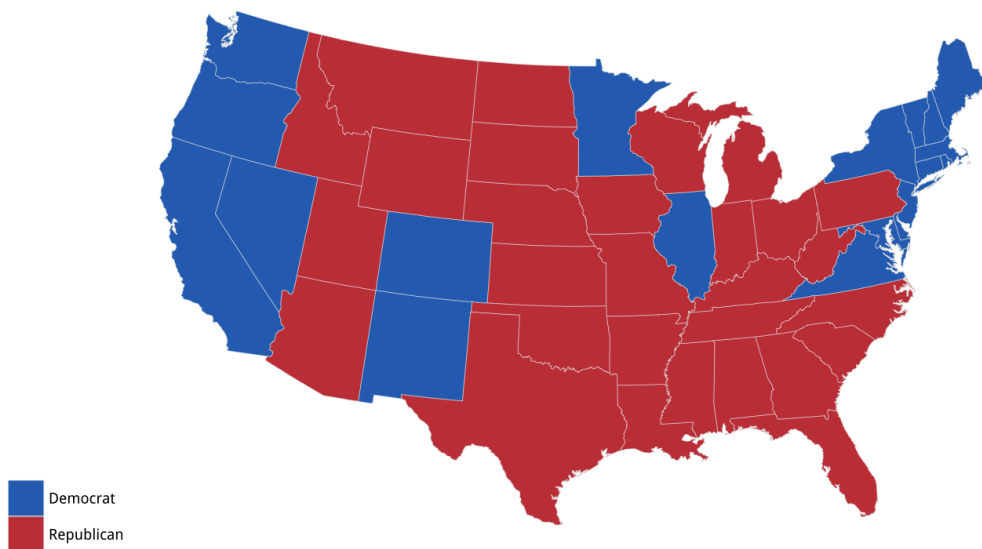


Political Science

R is used to analyse and visualise political data, including election results, public opinion surveys, and legislative voting records. Researchers can use R to create interactive maps showing election outcomes or to model political behaviour.

The following image is from a step-by-step tutorial on [how to draw choropleth maps in R](#).

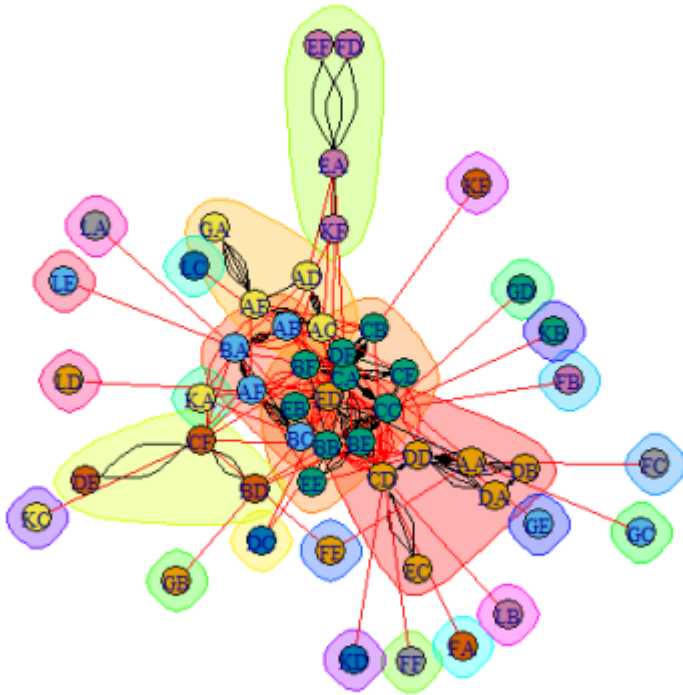
Election Results 2016



Sociology

Sociologists use R to analyse large-scale survey data, social network data, and demographic trends. R can help researchers identify patterns of social behaviour, study social networks, and analyse factors influencing social change.

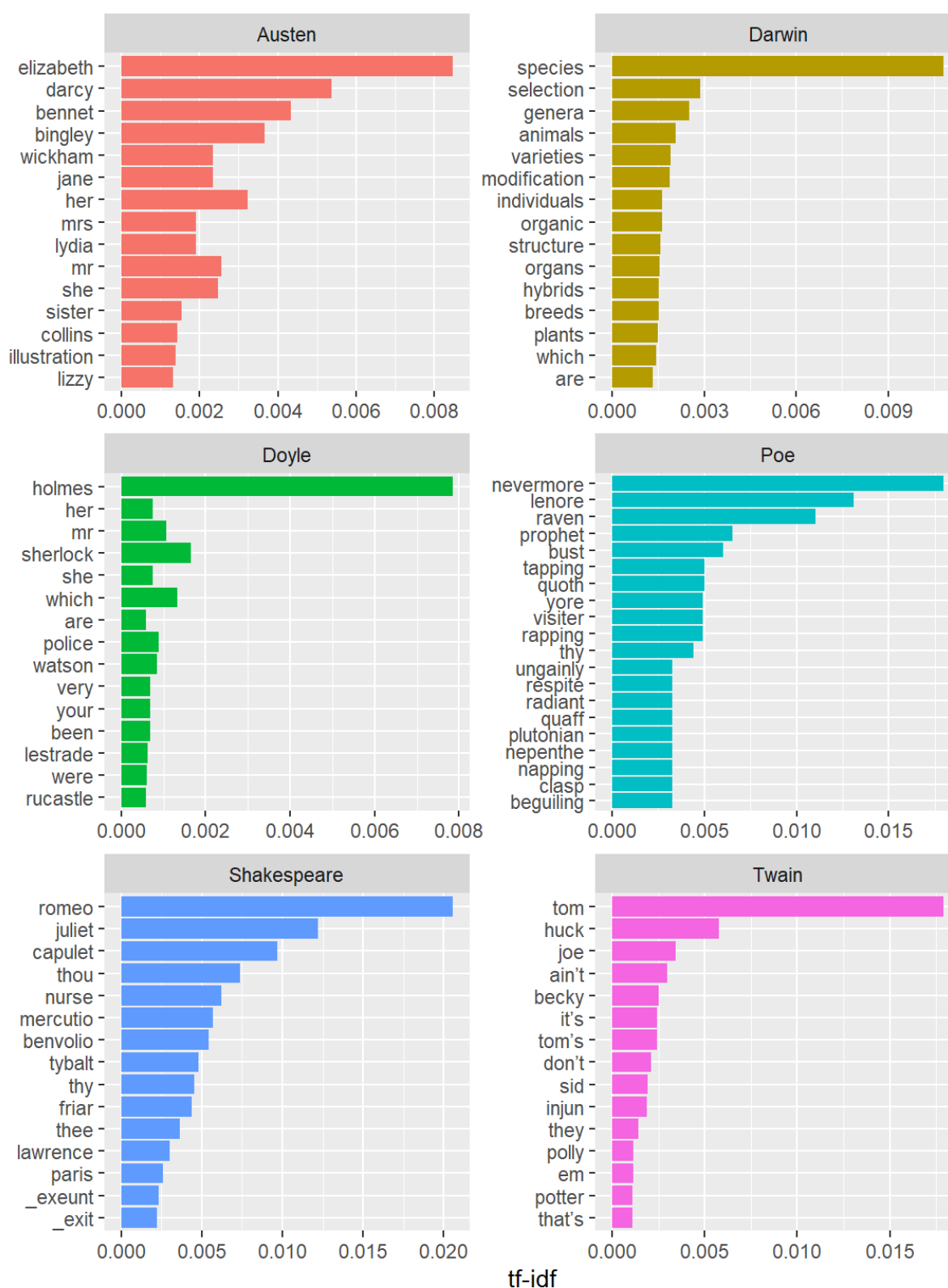
The following image is from a step-by-step tutorial on [social network analysis in R](#).



Cultural Studies

R can be used to analyse cultural texts such as literature, films, and music. Researchers can employ text mining techniques to study themes, sentiment, and linguistic patterns in cultural artifacts.

The following step-by-step tutorial shows [how fictional texts can be analysed by using computational means](#).



Linguistics

Linguists use R to analyse linguistic data, including phonetic transcriptions, text corpora, and language usage patterns. R can help researchers study language variation, language change, and the structure of languages.

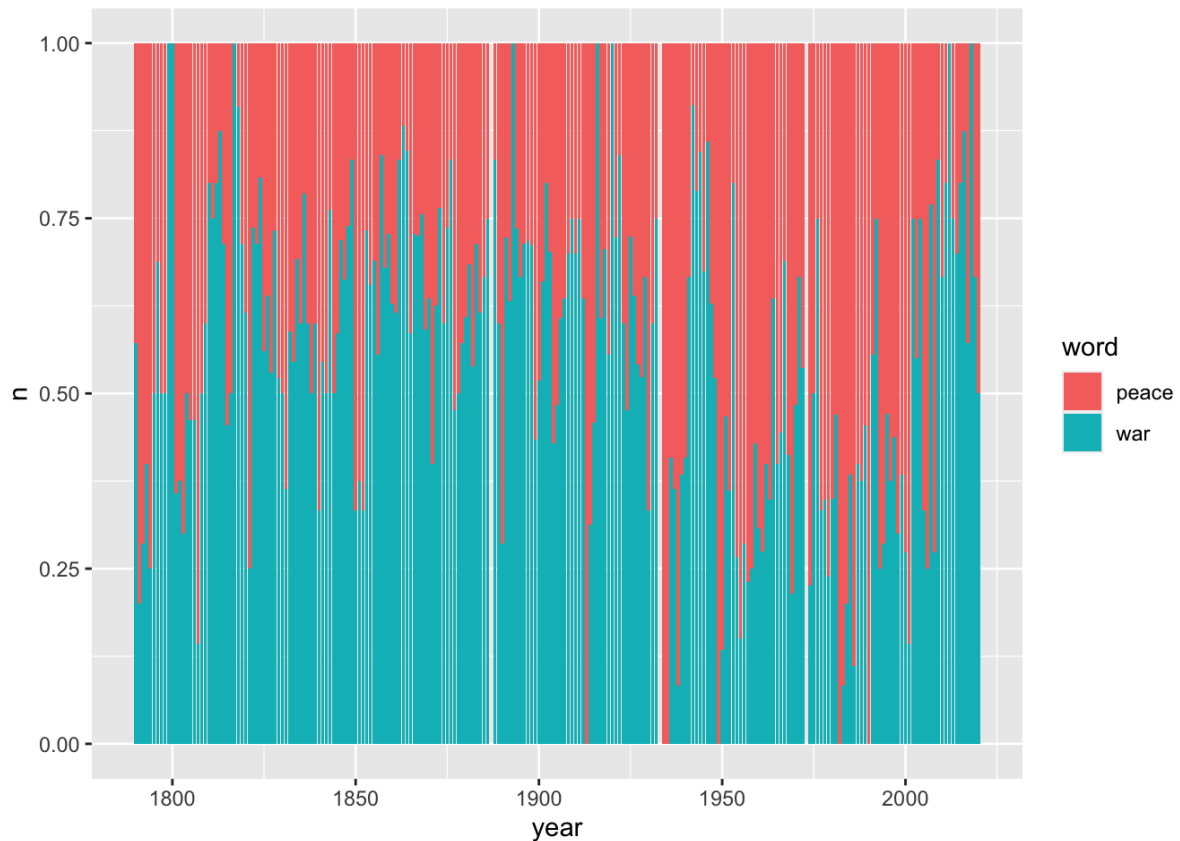
The following tutorial shows you [how to do text analysis in R](#).



History

R can be used to analyse historical data, including demographic records, archival documents, and historical maps. Researchers can use R to create visualisations of historical trends or to analyse patterns in historical texts.

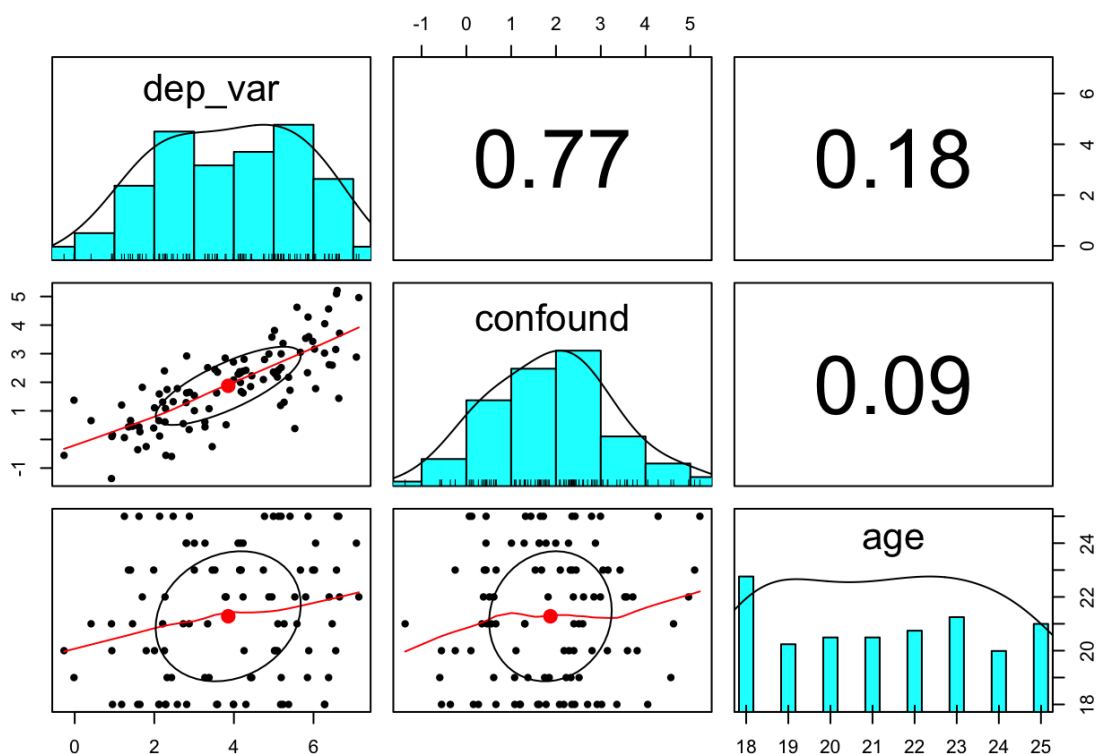
The following step-by-step tutorial shows you [how to do text analysis in R](#).



Psychology

R is commonly used for statistical analysis in psychology research, including experiments, surveys, and psychological assessments. Researchers can use R to analyse experimental data, conduct factor analyses, and create data visualisations.

The following tutorial shows you [how to use R for psychology research](#).



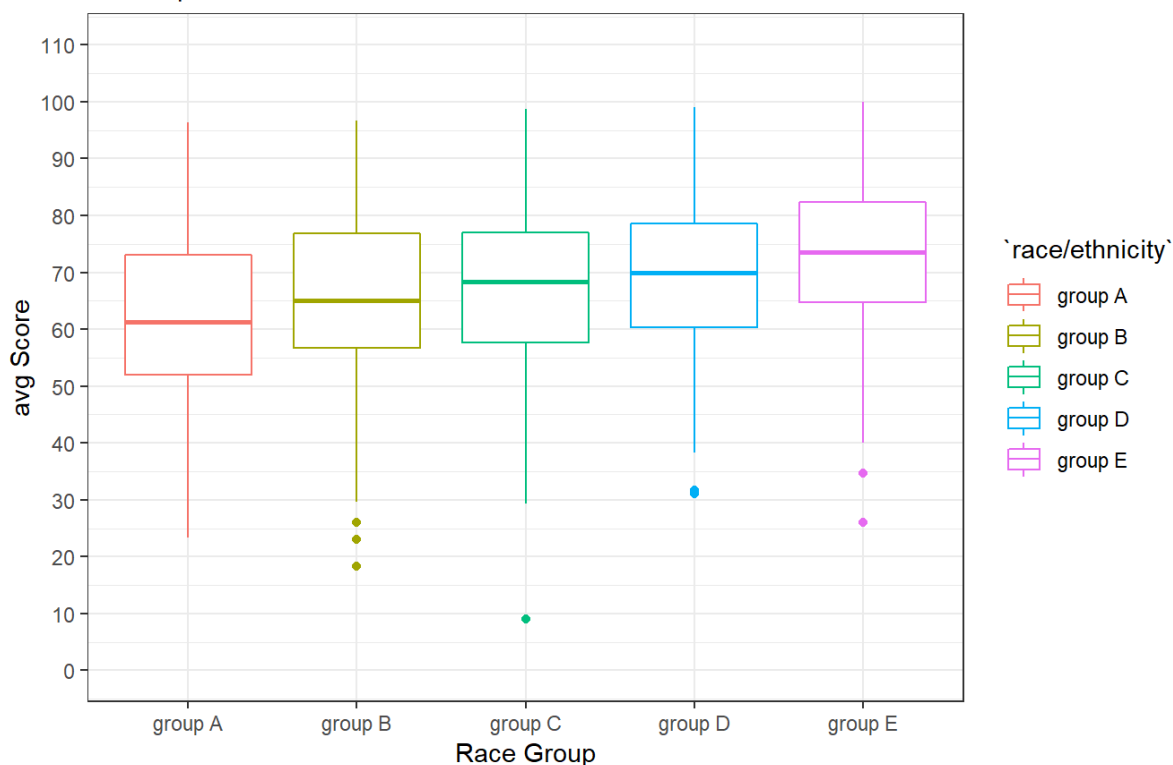
Education Research

R can be used to analyse educational data, including student performance data, assessment results, and educational interventions. Researchers can use R to evaluate the effectiveness of educational programs and interventions.

The following tutorial shows you how to do [student performance analysis in R](#).

The Urban Myth #3

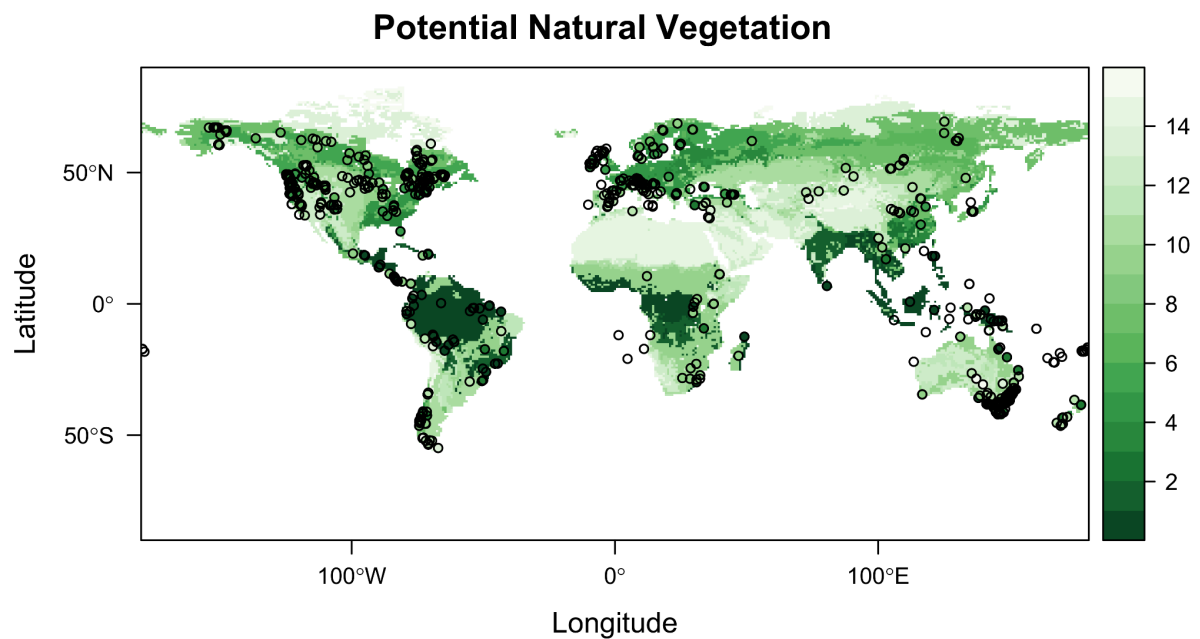
Does a particular race excels at math?



Geography

Geographers use R to analyse spatial data, create maps, and study geographical patterns. R can be used for geographic information system (GIS) analysis, spatial interpolation, and modelling of geographical processes.

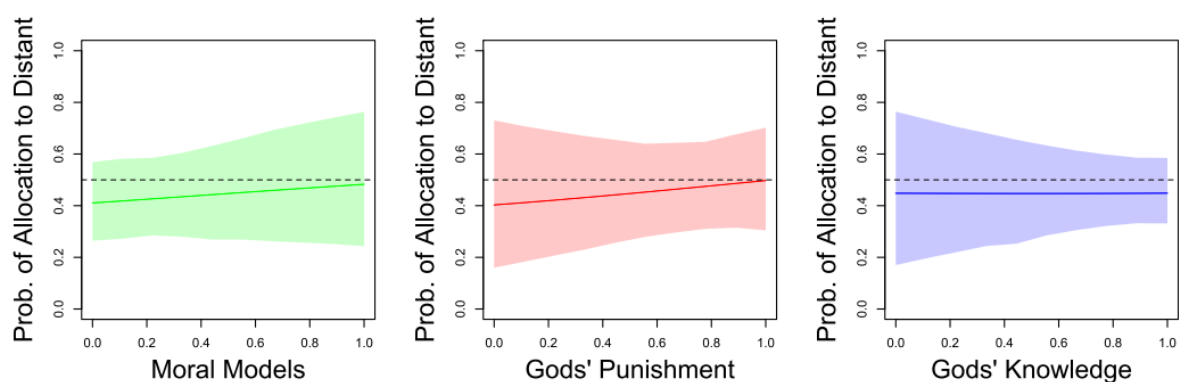
The following tutorial shows you [how to do geospatial analysis in R](#).



Anthropology

Anthropologists use R to analyse ethnographic data, cultural practices, and archaeological findings. R can help researchers analyse patterns of cultural diffusion, study kinship networks, and visualise spatial distributions of artifacts.

Visit the [AnthroTools](#) web page for documentation, R scripts, sources, and more.

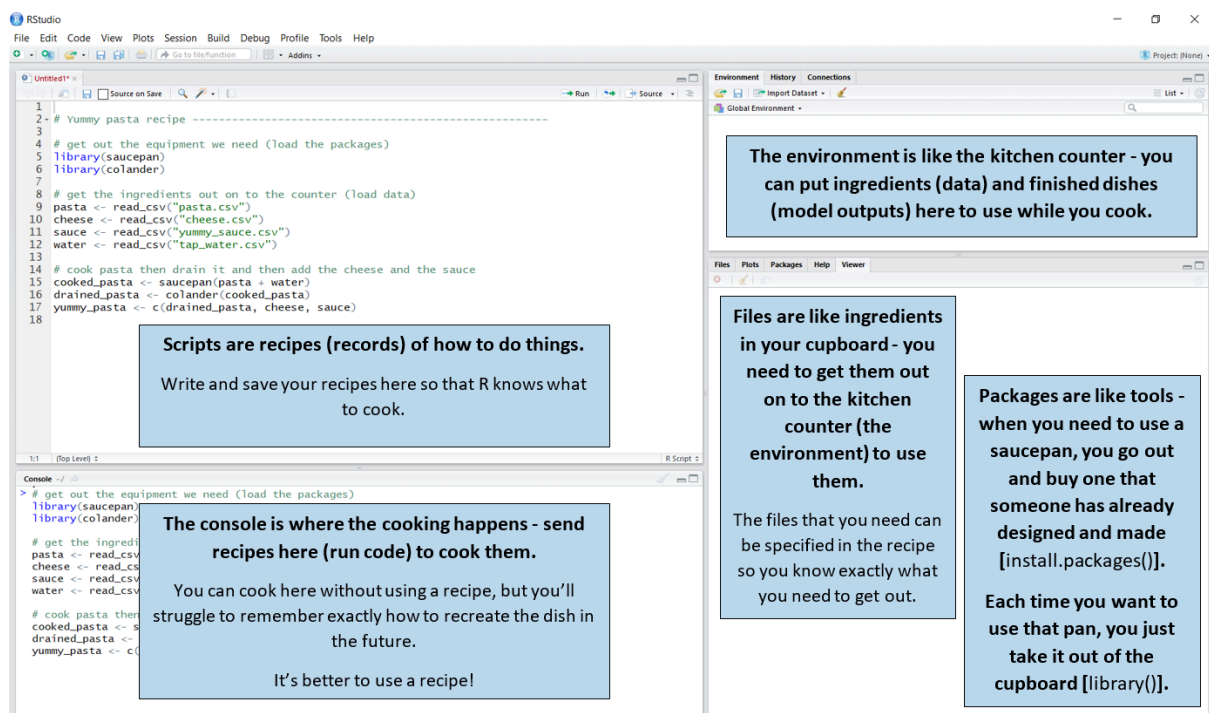


These are just a few examples of how the R programming language is applied in social sciences and humanities. R's versatility, extensive statistical capabilities, and data visualisation tools make it a valuable tool for researchers in these fields to conduct data-driven analyses and present their findings effectively.

Exploring the RStudio user interface

RStudio user interface

Let's use an analogy to understand how the different panes in RStudio work. Pretend that we want to make a pasta dish. There are two ways in which we can do this: with a recipe or without a recipe. How does this relate to the 4 panes in RStudio?



Source editor

The Source editor (top left pane) is used to write and save code as a script (i.e. recipes). We can reuse this script and modify it if we want to. These scripts can also be shared with colleagues so that they can review your code to make sure your analysis is correct, and they should be able to recreate the results.

Console

The Console (bottom left pane) is used to execute code. You can type code in the Console directly (i.e. cook pasta without writing down your recipe), but all the commands will be lost when you close RStudio. In other words, you will forget which ingredients and steps you used to create the pasta dish and you will be unable to reproduce it next time. Rather use the Source editor!

Environment

The Environment (top right pane) will store all the objects that you create. This is like our kitchen counter. We need to take out the ingredients (data) we need to make the pasta (calculation, model output or plot) and put it on the kitchen counter (Environment). These ingredients (objects) can be values, dataframes, plots, models, etc. If you want to analyse data, you will need to store the data as an object in the Environment, otherwise you will get an error that says 'Error: object not found'. In other words, you need to put the ingredients on the counter if you want to use them!



Note: this pane contains other tabs, such as History (where all the previous commands are stored) and Connections (where you can connect to external databases to access data).

Files and more

The Files pane (bottom right) contains multiple tabs:

- Files: see the contents of the project/working directory here, like your data and scripts.
- Plots: displays any plots that you create with code.
- Packages: view and/or load installed packages and install new packages.
- Help: search for specific functions to see how you should use it.
- Viewer: view HTML files (output from R Markdown documents).

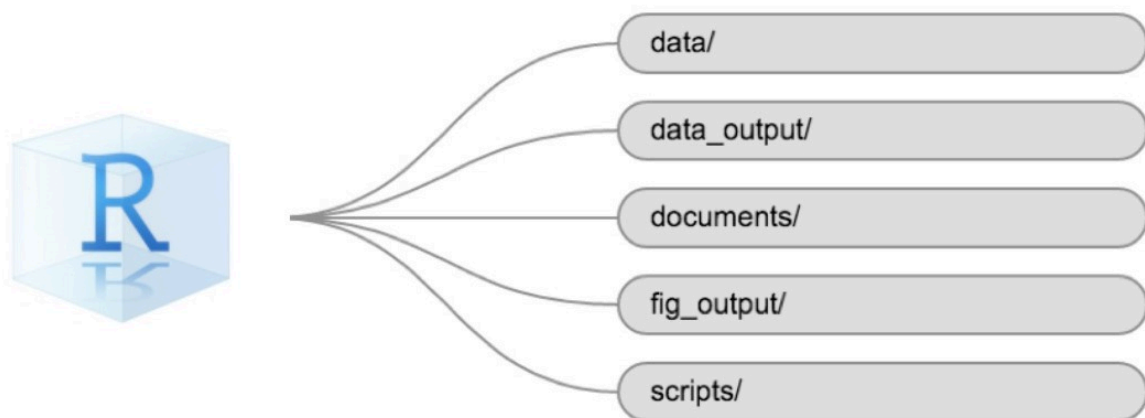
Create a reproducible project

Reproducible projects

What is a reproducible project?

A reproducible project refers to a systematic approach to organising and documenting your code, data, and analysis steps in a way that allows others (or even your future self) to easily understand, replicate, and verify your results. The goal of a reproducible project is to ensure that anyone who interacts with your code and data can obtain the same outcomes, insights, and conclusions as you did, following a well-defined and transparent process.

In general, you might create directories (folders) for scripts, data, and documents. Here are some examples of suggested directories:



- **data/** Use this folder to store your raw data and intermediate datasets. For the sake of transparency and **provenance**, you should *always* keep a copy of your raw data accessible and do as much of your data cleanup and preprocessing programmatically (i.e., with scripts, rather than manually) as possible.
- **data_output/** When you need to modify your raw data, it might be useful to store the modified versions of the datasets in a different folder.
- **documents/** Used for outlines, drafts, and other text.
- **fig_output/** This folder can store the graphics that are generated by your scripts.
- **scripts/** A place to keep your R scripts for different analyses or plotting.

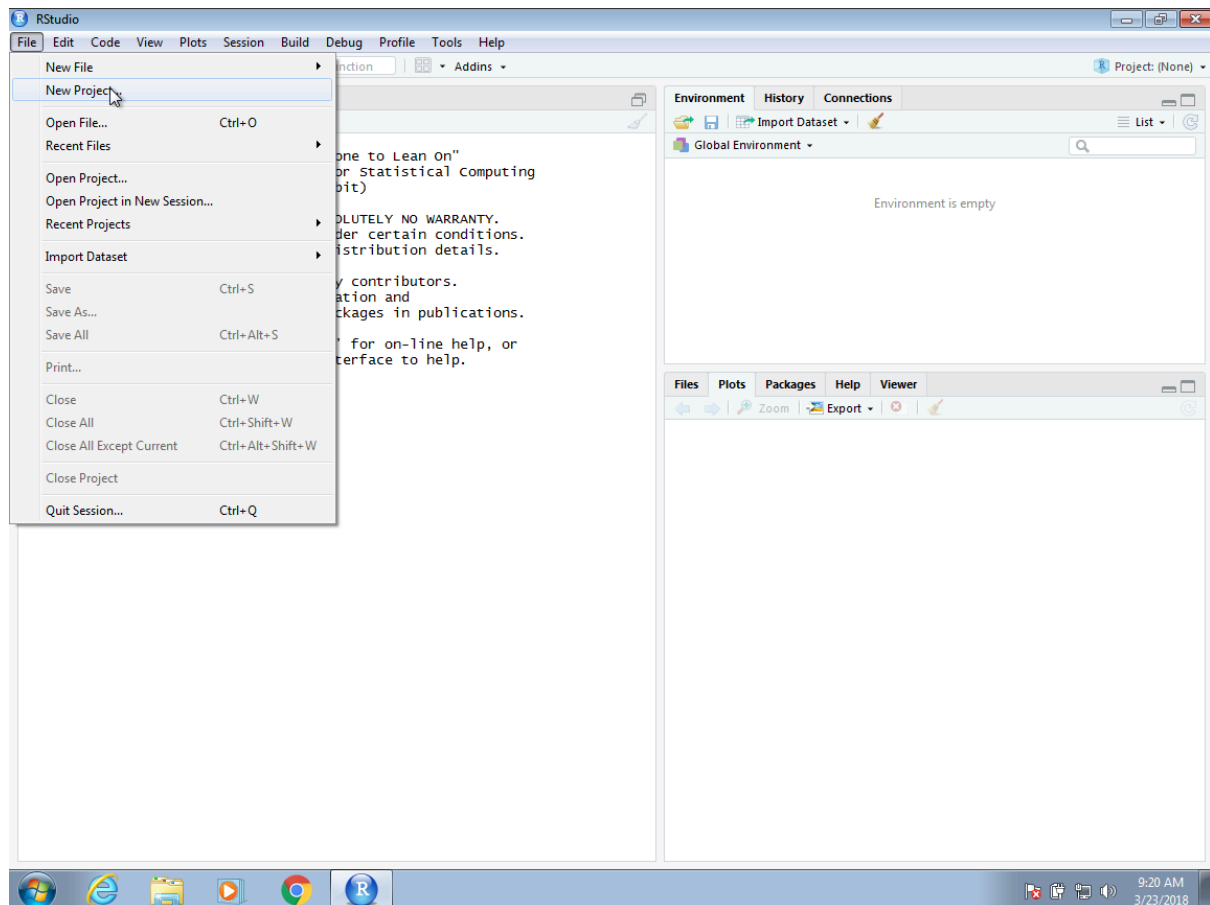
You may want additional directories or subdirectories depending on your project needs, but these should form the backbone of your working directory 💡.

💡 A working directory is the folder that contains all the subfolders and files of your project. This is where R will look for files that you ask it to load, and where it will put any files that you ask it to save.

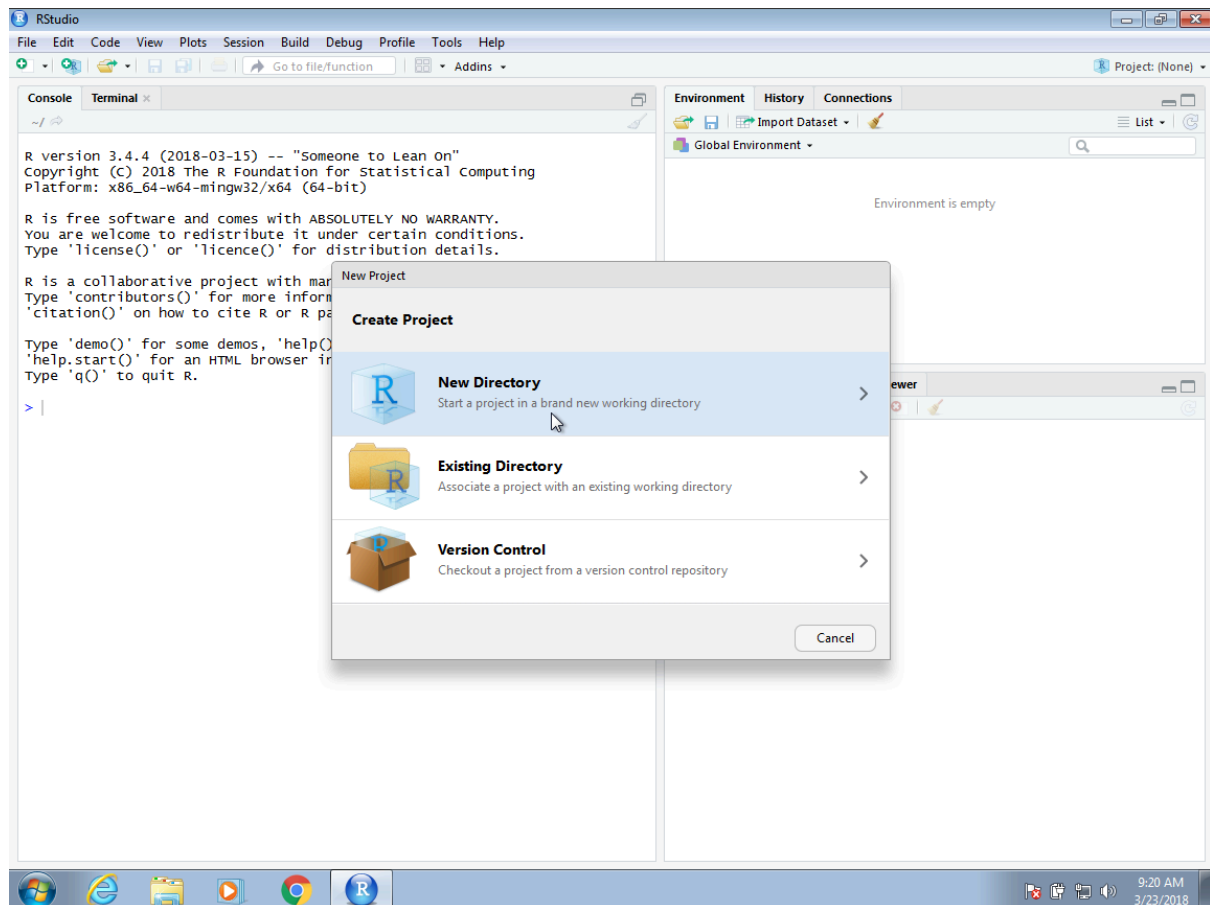
How do I create a reproducible project?

Certain sections of the following step-by-step tutorial is from the book [Introduction to Data Science](#), written by Rafael A. Irizarry.

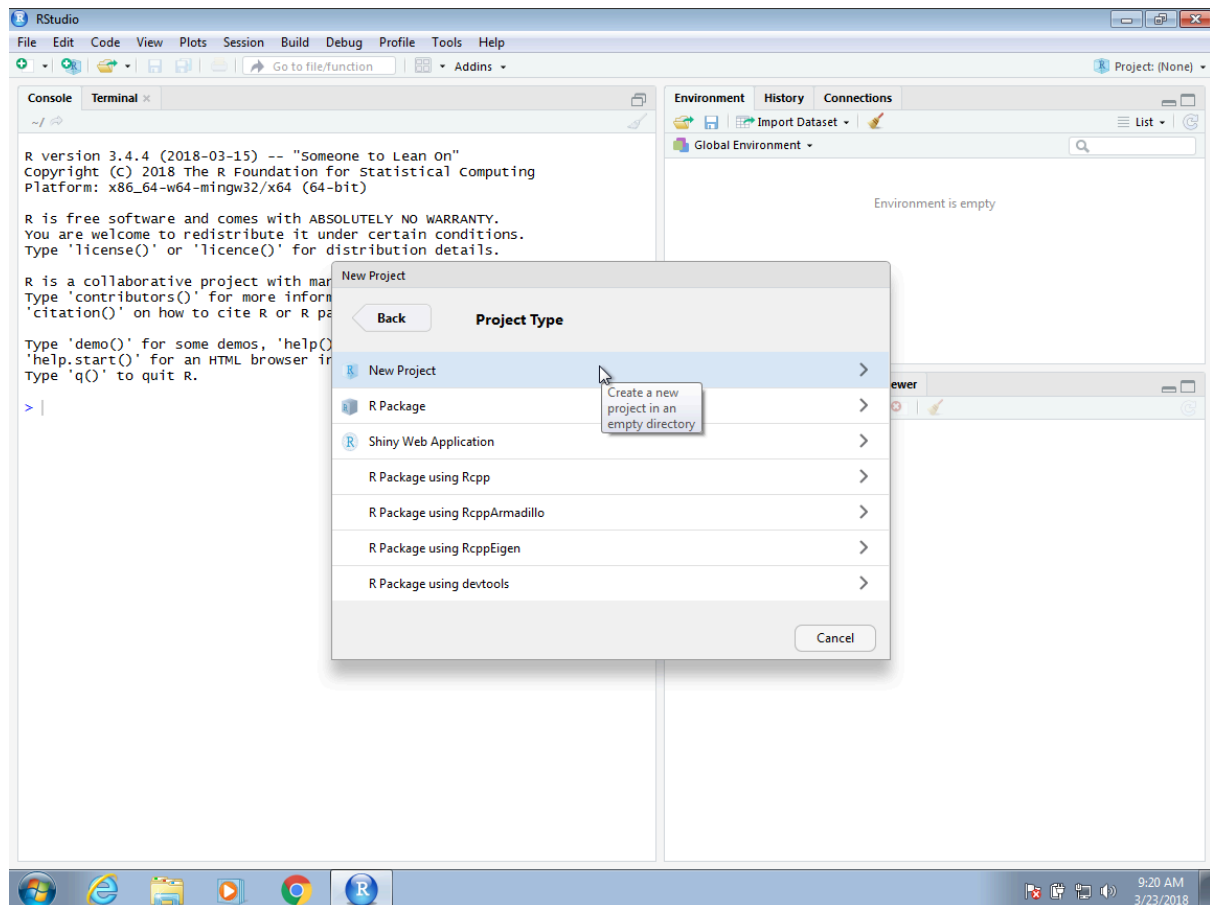
To start a project, click on *File* and then *New Project*.



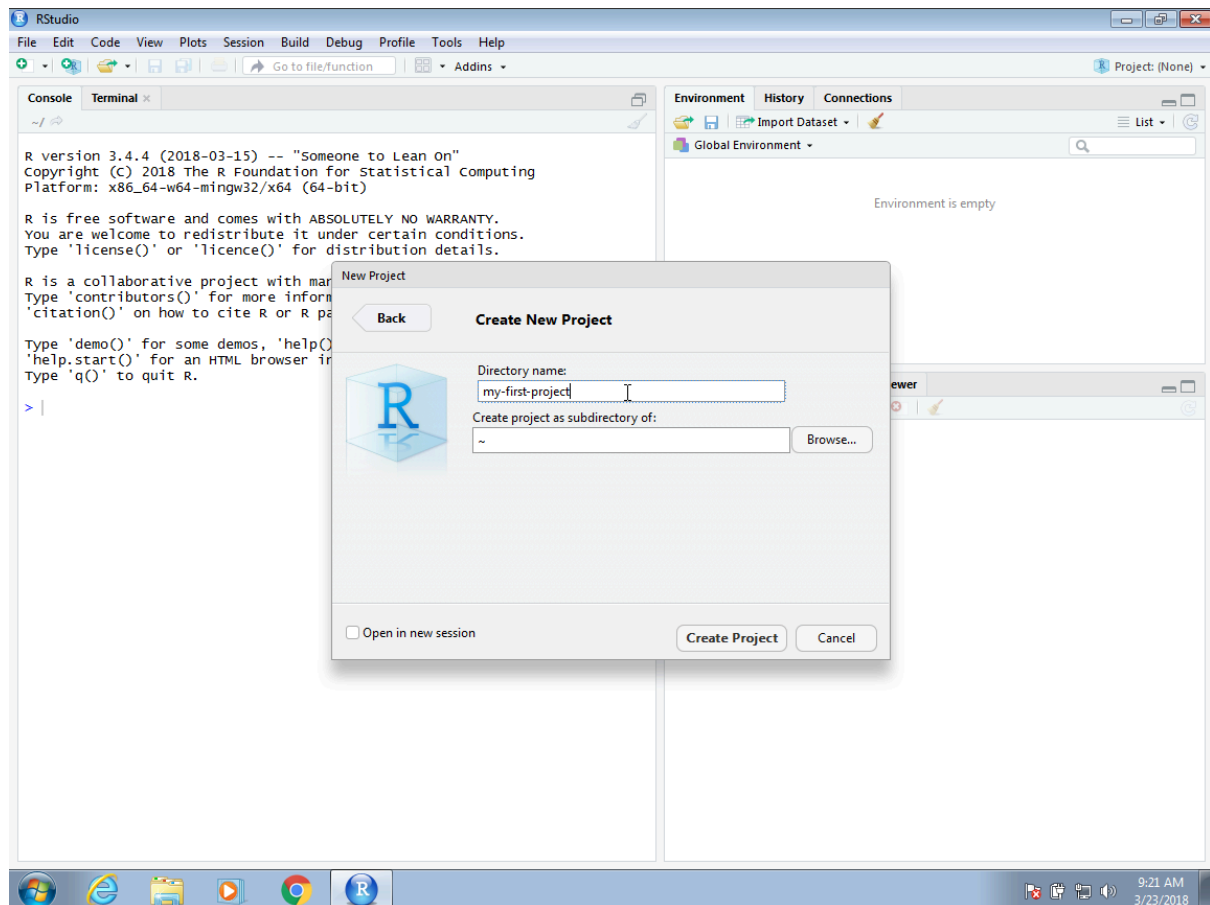
If you already have an existing folder for your project, select *Existing Directory*. Alternatively, choose *New Directory* to create a new folder for your project.



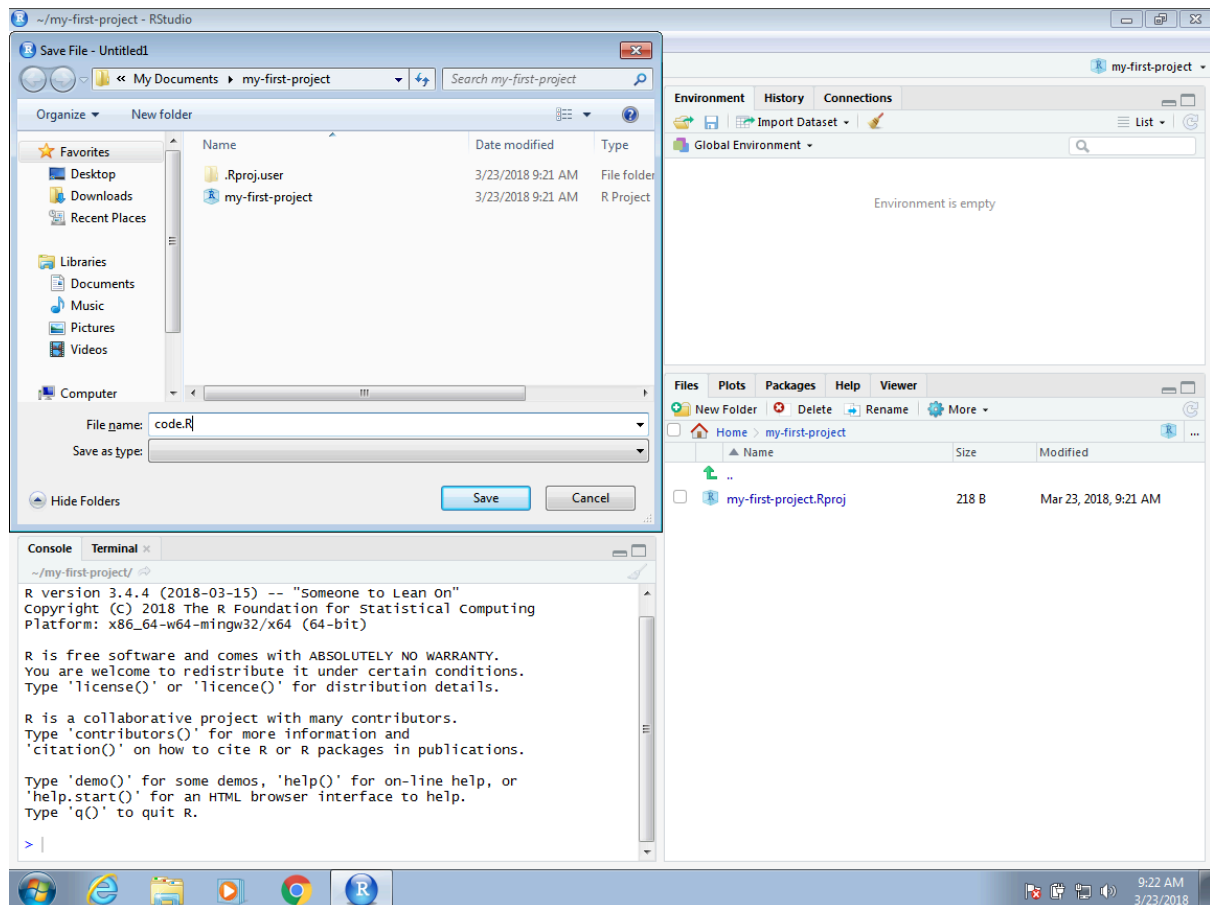
For a data analysis project, select the *New Project* option:



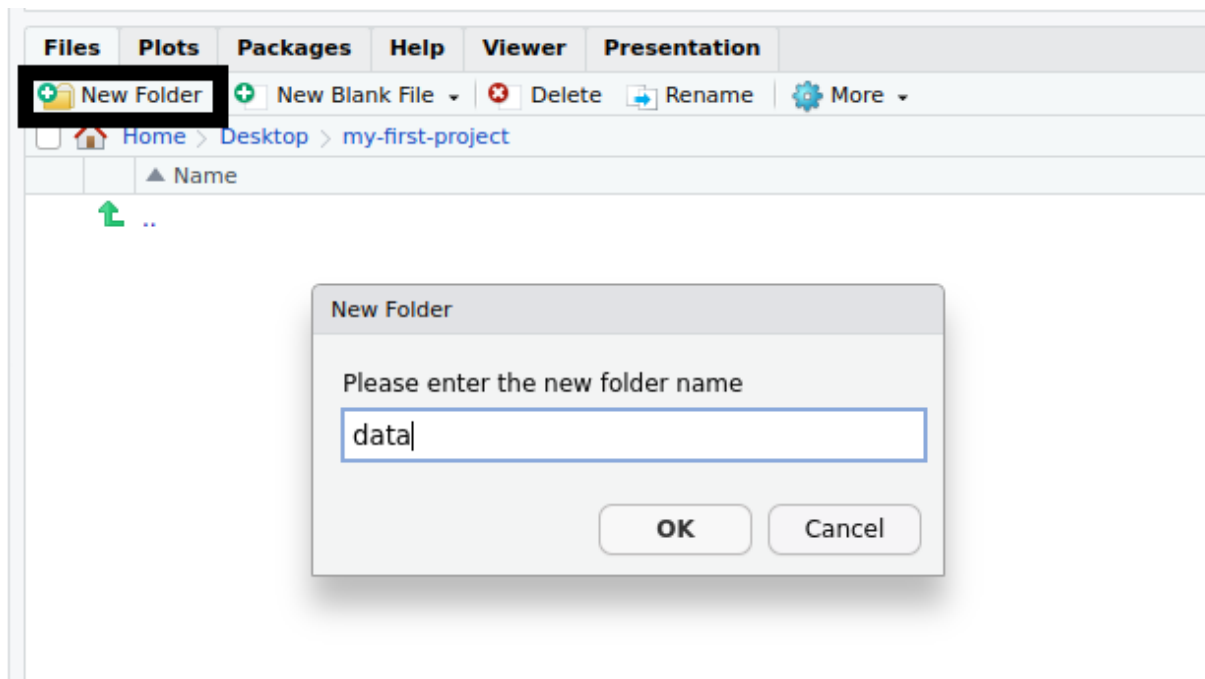
Next, type a name for this new folder name. Make sure it is a meaningful name that will help you remember what the project is about. As with files, we recommend using lower case letters, no spaces, and hyphens to separate words. We will call the folder for this project *my-first-project*. This will then generate a *Rproj* file called *my-first-project.Rproj* in the folder associated with the project. We will see how this is useful a few lines below.



Click *Browse* and navigate to the location on your file system where you would like to save this new project. In the following example, we are storing our project (my-first-project) inside the *My Documents* folder.

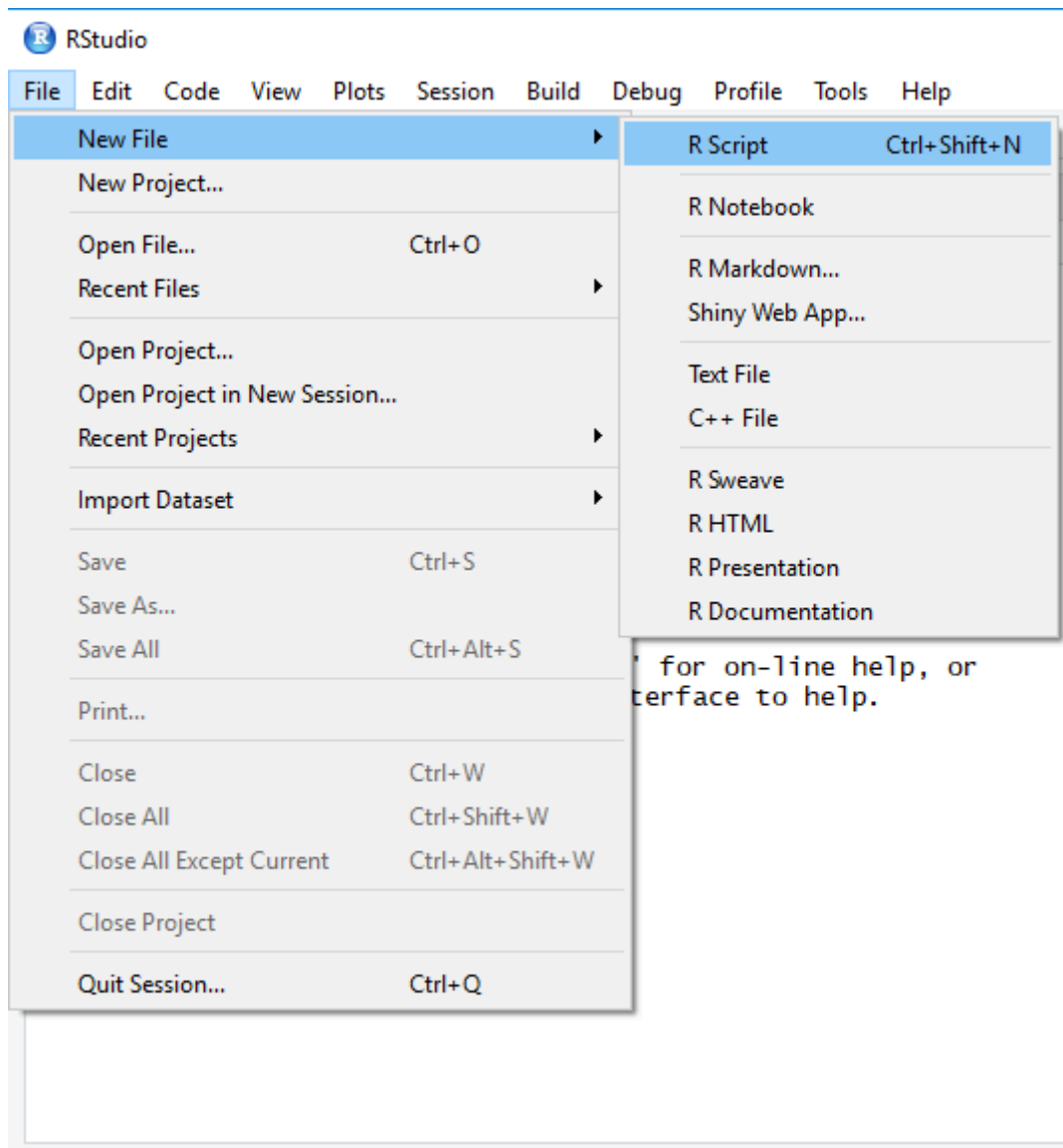


Now it's time to create all the folders we will need for storing our data, scripts, and documents. To create a new folder, click *New folder* and type a name for the folder (e.g. data) and click OK.



Repeat this step to create the following two folders: data_output and fig_output.

Once you've created all the folders you will need, go ahead and create a new script by clicking on *File > New File > R Script*.



To save your script, click the  Save icon at the top of the script. Type a name for this script and browse to the scripts folder and click Save.

Now you have a reproducible project!

One of the main advantages of using Projects is that after closing RStudio, if we wish to continue where we left off on the project, we simply double click or open the file saved when we first created the RStudio project. In this case, the file is called *my-first-project.Rproj*. If we open this file, RStudio will start up and open the scripts we were editing. You can also share this project with colleagues, and they will be able to run the analyses and produce the same results.

Week 2: Using existing code to save time and be more efficient

Our learning goals for this week

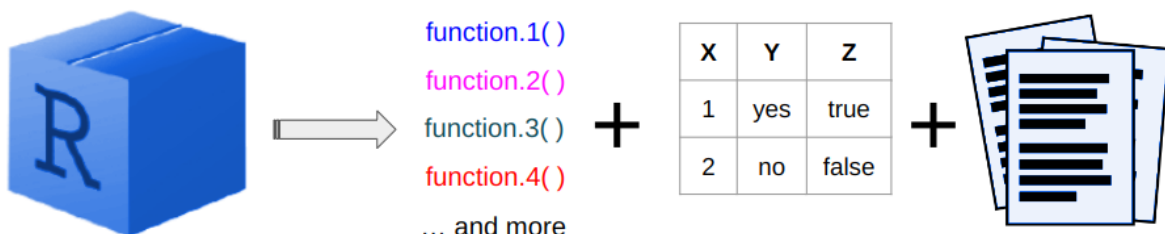
1. Describe the difference between packages and functions
2. Install and use packages and functions
3. Access open data
4. Give examples of resources and communities to find help with programming
5. Know how to find out if someone has already published a function that you can re-use

Difference between packages and functions

Packages and functions

Packages

R packages are a collection of R functions, sample data, and documentation.



A single package can contain 1 to 100 functions! By default, R installs a set of packages during installation. However, if you need more packages, you will need to install them. The complete list of published packages are available on the [CRAN \(Comprehensive R Archive Network\)](https://cran.r-project.org/) website.

Functions

A function is a set of instructions that you run on the input (i.e., the data) to return the desired output.

A function looks like this:

function(argument1, argument2, ...)

For example, the `round()` function will take a numeric value with decimals and round it either to zero digits (default) or to the number of digits specified by you. Let's break it down:

- Firstly, you have to type the name of the function, followed by rounded brackets, e.g. `round()`.

 Note: there is no space between the name of the function and the rounded brackets.

- Some functions also take arguments, which are specified within the rounded brackets, e.g. `round(x = 3.142, digits = 2)`. The arguments change the way the function works, e.g. if you specify `round(x = 3.142, digits = 2)`, then the `round()` function will round the input value (`x`) to 2 digits.

Install and use packages and functions

Packages and functions

In order to use specific functions from a specific package, you need to install the package first. A package is only installed once, but you have to load the package every time you start a new R session. It's like buying a pot to make pasta: you only buy the pot once, but you need to take it out of the cupboard every time you want to make pasta!

To install a package from [CRAN](#), you can either do it via command line in the console or make use of the RStudio IDE.

Install a package via command line

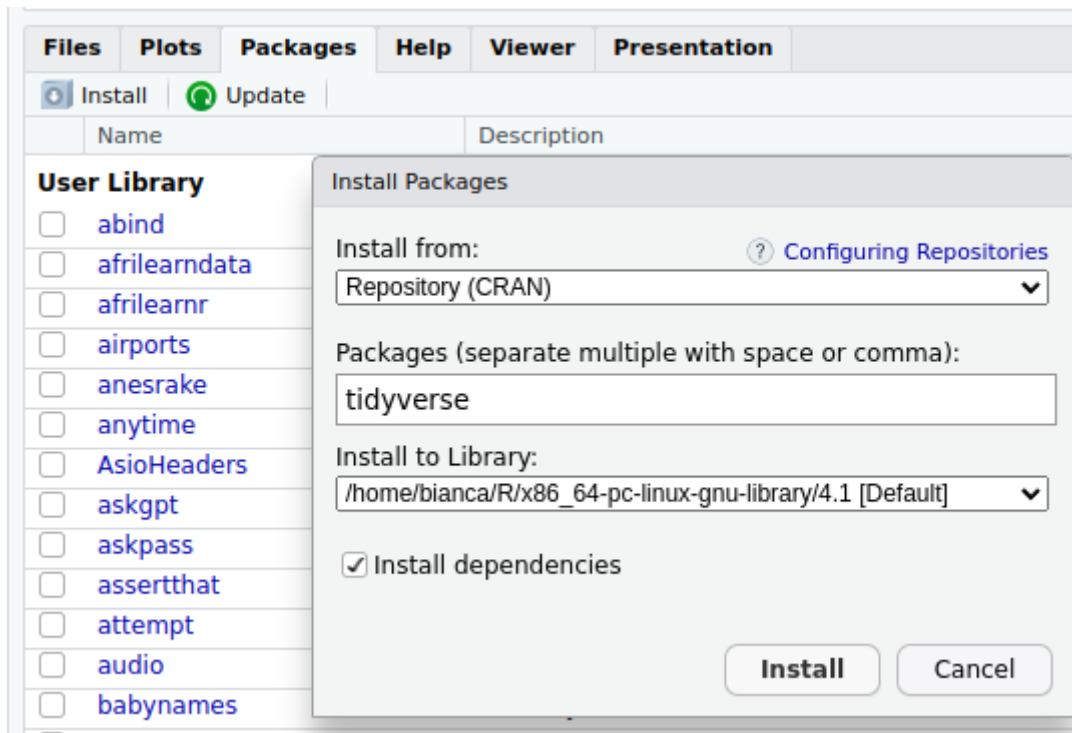
In the Console, type `install.packages("<name>")` and press *Enter* to execute this command. Replace `<name>` with the name of the package, for example `install.packages("tidyverse")`.

 Note: The name of the package should be in quotes when installing the package.

Install a package via the IDE

In the Fields and more pane:

1. Click on the *Packages* tab, then click *Install*.
2. Type the name of the package you want to install, e.g. *tidyverse*, then click *Install*.



Load a package

After you have installed a package, you need to load the package in order to use the functions inside that package. To load the package, type the following code in the Console and press *Enter*: `library(<name>)`, where `<name>` should be the name of the package, e.g. `library(tidyverse)`.

 Note: The name of the package does not need to be in quotes when loading the package, but it will still work with quotes, e.g. `library("tidyverse")`.

Exercise

Install and load the `{wordcloud}` package with the following commands:

```
install.packages("wordcloud")
```

```
library(wordcloud)
```

Use a function

Let's refresh our memories on what a function looks like:

function(argument1, argument2, ...)

- Firstly, you have to type the name of the function, followed by rounded brackets, e.g. `round()`.

 Note: there is no space between the name of the function and the rounded brackets.

- Some functions also take arguments, which are specified within the rounded brackets, e.g. `round(x = 3.142, digits = 2)`. The arguments change the way the function works, e.g. if you specify `round(x = 3.142, digits = 2)`, then the `round()` function will round the input value (x) to 2 digits.

 You can use the function without typing out the names of the arguments, e.g. `round(3.142, 2)`, but then the arguments should be in the correct order, otherwise the function might not work as expected. It is best practise to type out the names of the arguments, e.g. `x = 3.142, digits = 2`.

Access open data

Access open data

Built-in data sets

There are many data sets available in base R and in different packages of R. The characteristics of these data sets are very different, for example, some data sets are time series data, some have only numerical columns, some have numerical as well as factor columns, some include character columns together with other types of columns.

To get the list of available data sets in base R we can use the `data()` function, but to get the list of data sets available in a package we first need to load that package with the `library()` function, and then use the `data()` command to show the available data sets in that package.

Example 1:

`data(iris)` -> load the iris data set in the package `{datasets}`

Example 2:

`library(ggplot)` -> to load the `{ggplot2}` package

`data()` -> to view the data sets in the `{ggplot2}` package

If you want to use a specific data set from the `{ggplot2}` package, use `data(<name>)` and replace `<name>` with the name of the data set.

Download data via a URL

If, for example, there is a CSV file on [figshare](#) that you'd like to download and load into R's memory, you will need to do the following:

1. Navigate to the specific CSV file on figshare, then find the  download button, right-click on it, and choose `Copy link address`.
2. In RStudio, use the `download.file()` function to download the file:

```
download.file(url = "https://ndownloader.figshare.com/files/11492171",  
              destfile = "data/SAFI_clean.csv",  
              mode = "wb")
```

Let's break it down to understand what these arguments mean:

1. `url` -> the URL to the data set that you copied to the clipboard previously.
2. `destfile` -> the destination of the file, i.e., where you would like to store the data.
In this case, it will be stored in the `data/` folder.
3. `mode` -> `"wb"` will open a write-only file in binary mode.

Re-use published functions

There are thousands of functions in the R programming language and more functions are added to the [Comprehensive R Archive Network \(CRAN\) homepage](#) every day.

[Statistics Globe](#), an education platform that provides free programming tutorials, has compiled a list of the [most popular and useful R functions](#). Click on a specific function and follow the tutorial with quick examples to learn how this function works.

The [R Graph Gallery](#) also provides multiple examples of visualisations and the code that was used to produce these visualisations.

If you want to re-use this code for your work, you can simply copy and paste the code into your R script. You will then need to substitute the variable names in the *example code* with the variable names that are in your *data set*. For example, the code on the internet shows you how to create a basic scatter plot with the following code: `plot(x, y)`

If you have a dataframe in the global environment called `surveys`, and you have two continuous variables called `weight` and `height`, you would need to modify the example code as follows: `plot(surveys$weight, surveys$height)`

Week 3: Working with tabular data

Our learning goals for this week

1. Load and inspect tabular data
2. Create a wordcloud
3. Give examples of resources and communities to find help with programming

Load and inspect tabular data

Get the data

Let's download a dataset to work with: [frequency_list.csv](#)

This data is a subset of the NCHLT English Text Corpora that you can download from the [SADiLaR Resource Catalogue](#).

This data is a collection consisting of a clean corpus, lexicon, frequency list and named-entity lists developed during the NCHLT Text project.

Load data

There are two ways to load data in RStudio:

- Via the interface
- Via the commandline

1. Load data via the interface

- Option 1: Through the RStudio menu click File > Import Dataset > then choose one of the following options:
 - If it's a CSV file, choose From Text (base)... or From Text (readr)...

- If it's an Excel file, choose From Excel...
- Option 2: From the Environment pane, use the Import Dataset dropdown menu, then choose one of the options specified above.

After choosing the data file, look at the data preview and click Import. The command to import the data will be displayed in the console, and you can copy and paste that command into the Script editor to use it in future.

2. Load data via the command line

- Firstly, you need to copy the data file into your project folder. If we use the folder structure we created previously, put the data into the `data/` folder.
- In the Script editor, type and run the following command: `data <- read_csv("data/<name>.csv")` and replace `<name>` with the filename.

Inspect data

There are several commands we can use to inspect the data:

- `View(data)` -> displays the data as a table
- `str(data)` -> indicates the structure of the data
- `dim(data)` -> shows the dimensions of the data
- `head(data)` -> shows the first 6 rows of the data
- `tail(data)` -> shows the last 6 rows of the data
- `colnames(data)` -> shows the column (variable) names of the data
- `rownames(data)` -> shows the row names of the data
- `summary(data)` -> shows descriptive statistics of the data

Create a wordcloud

Now that you have downloaded and loaded your data, you can apply many other functions to this dataframe object.

One fun thing that you can do with word frequency data like this, is create a word cloud.

Create a word cloud

We will need quite a few lines of code to create this word cloud.

Here are the steps:

1. First install and load the `{readr}` and `{wordcloud}` packages.
2. Load the data from the CSV file and create a "data" object using the `read_csv()` function from the `{readr}` library:

```
data <- read_csv(<filename>.csv)
```

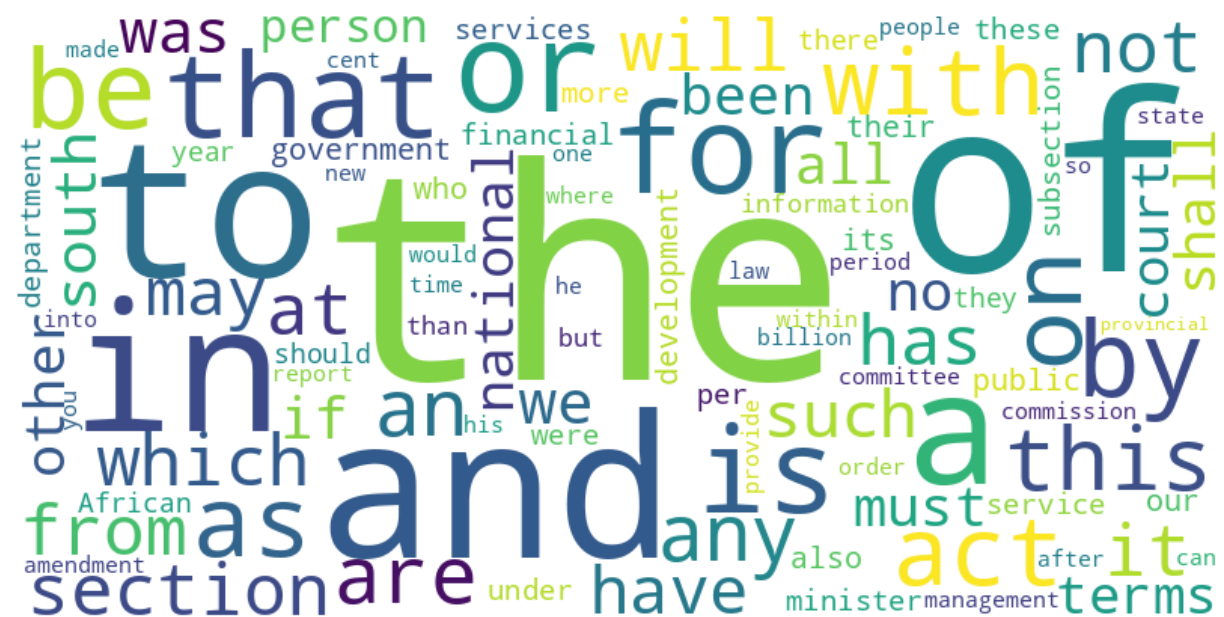
3. Create a "wordcloud" object using the `wordcloud()` function from the `{wordcloud}` library:

```
wordcloud <- wordcloud(words = data$word, freq = data$frequency)
```

5. Save the image as a PNG file by clicking on the Export button in the Viewer pane (bottom-right).

Expected output

In the end, your word cloud image will look like this:



Of course this is not the only way to create a word cloud and save the image file - there are many other methods and libraries that you can use.

Resources

Here are some other resources that you might find useful:

- The [wordcloud](#) package [documentation](#) and a [tutorial](#).

Resources and communities to find help with programming

Interactive learning

If you are interested in learning R in an interactive way, try the following:

1. Explore the [{swirl}](#) package via the RStudio console:
 2. Work through the [RStudio Primers](#) online platform.
- Install the [{swirl}](#) package:

```
install.packages("swirl")
```

- Load the [{swirl}](#) package:

```
library(swirl)
```

- Follow the on-screen instructions to complete the tutorial.

Videos, webinars and blogs

- If you prefer learning through instruction, have a look at the [free videos and webinars](#) on Posit's website.
- If you prefer to read, look at [The Posit blog](#).

RStudio education

If you're unsure where to start, choose your learning path from the [RStudio education site](#) to help you become productive in the minimum amount of time, based on your experience level.

Examples with visualisation and code

- [RGraph gallery](#)
- [RStudio primers](#)

Cheat sheets

Download [cheat sheets](#) for free and keep them next to your computer to remind you of the various functions that you can use for data wrangling and visualisation.

Books

There are many [open source books](#) available. I would suggest that you start with "R for Data Science" and move onto the more advanced topics from there.

A gentle introduction to tidy statistics in R

This [tutorial](#) covers the basics of research statistics in R, providing an emphasis on reading data into R, exploratory data analysis with the [Tidyverse](#), statistical testing with ANOVAs, and finally producing a publication-ready plot in [ggplot2](#).

Communities

Join the following communities or forums:

- ESCALATOR Workspace:
 - To join the workspace, click [here](#).
 - To go to the workspace if you have already joined, go to dhcssza.slack.com.
- Click on [R for Data Science Online Learning Community](#) to join their Slack workspace. You can ask questions about anything that is related to R.
- Look at the [Posit Community](#) for previously asked questions, or post a new question to get help from experts. It's important that you provide a [reproducible example](#) when you ask a question.

Week 4: Best practices in programming

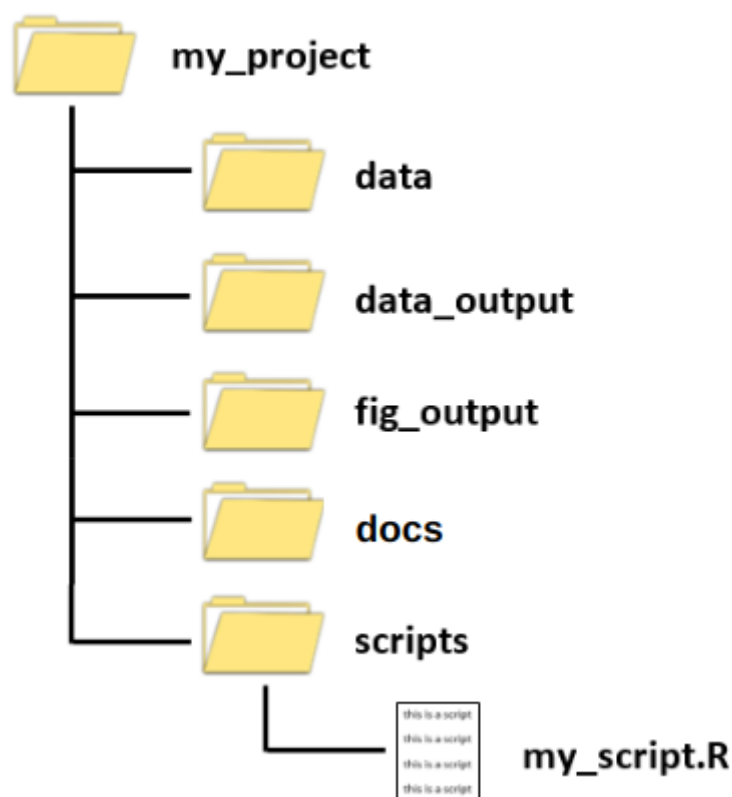
Our learning goals for this week

1. Give examples of best practices in programming
2. Make code re-useable and decipherable (commenting)
3. Recognise good naming conventions
4. Understand the first steps to reading error messages
5. Identify possible next steps to continue learning to programme

Examples of best practices in programming

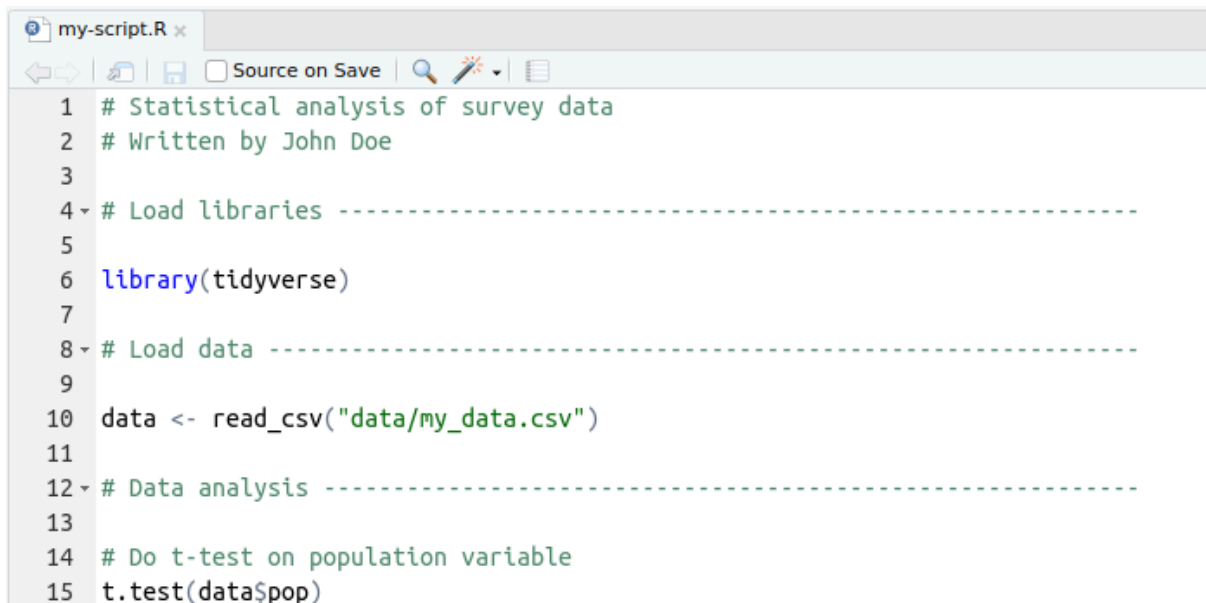
File organisation

- Create a reproducible project and organise files in a sensible way within the project:
 - **data**: this folder contains the raw data and should never be modified.
 - **data_output**: this folder stores any intermediate and final datasets that were generated via code during analyses.
 - **fig_output**: all the visualisations produced during the analyses are stored in this folder.
 - **docs**: store any documents related to your project, such as notes, manuscripts, etc.
 - **scripts**: save all the scripts with code in this folder.



- Organise code within each script:
 - Start each file with a comment saying who wrote it and when, what it contains, and how it fits into the larger program.
 - Then load all required packages.
 - Then source (load) required files (if any).
 - Then your code starts.

- If you have a lot of code or different analyses, break code up into separate files (generally <2000–3000 lines).



```
1 # Statistical analysis of survey data
2 # Written by John Doe
3
4 # Load libraries -----
5
6 library(tidyverse)
7
8 # Load data -----
9
10 data <- read_csv("data/my_data.csv")
11
12 # Data analysis -----
13
14 # Do t-test on population variable
15 t.test(data$pop)
```

Assignment

Use `<-` (inserted with the **Alt** - shortcut) instead of `=`.

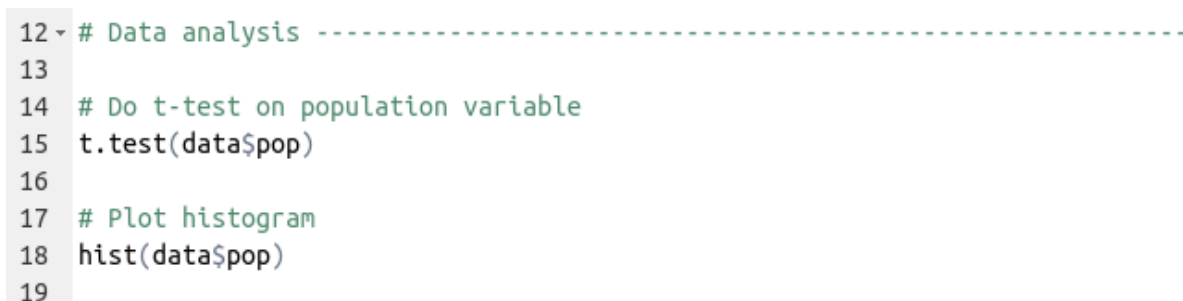
Examples:

😞 `mydata = read_csv("surveys.csv")`

😄 `mydata <- read_csv("surveys.csv")`

Comments

Use comments to mark off sections of code, and also to explain the purpose of the various code blocks.



```
12 # Data analysis -----
13
14 # Do t-test on population variable
15 t.test(data$pop)
16
17 # Plot histogram
18 hist(data$pop)
19
```

Naming conventions

- Generally, variable names should be nouns and function names should be verbs, e.g. `country` can be used as a variable name in your dataset, and `calculate_birth_rate` can be used as a function name.
- Use names that are concise and meaningful, e.g. `pop_counts` (for population counts) is better than using a single letter such as `x`.
- Names of object or functions should not start with numbers.
- It's generally easier to write everything in lower case, but you can also use periods (e.g. `plot.new`), underscores (e.g. `plot_new`), or camel case (e.g. `PlotNew`) to name objects or functions.

Relative paths

Always use relative paths in your scripts to ensure that it will run successfully on any other computer.

Example:

😞 `mydata = read_csv("C:/Users/John/PhD/project/data_raw/surveys.csv")`

😄 `mydata <- read_csv("project/data_raw/surveys.csv")`

Code review

It's always a great idea to have someone else review your code. This is easy when you have created a reproducible project and used the above best practices.

The tidyverse style guide

If you want to read more about good coding style, have a look at the [tidyverse style guide](#).

Make code re-usable and decipherable

Best practices in programming

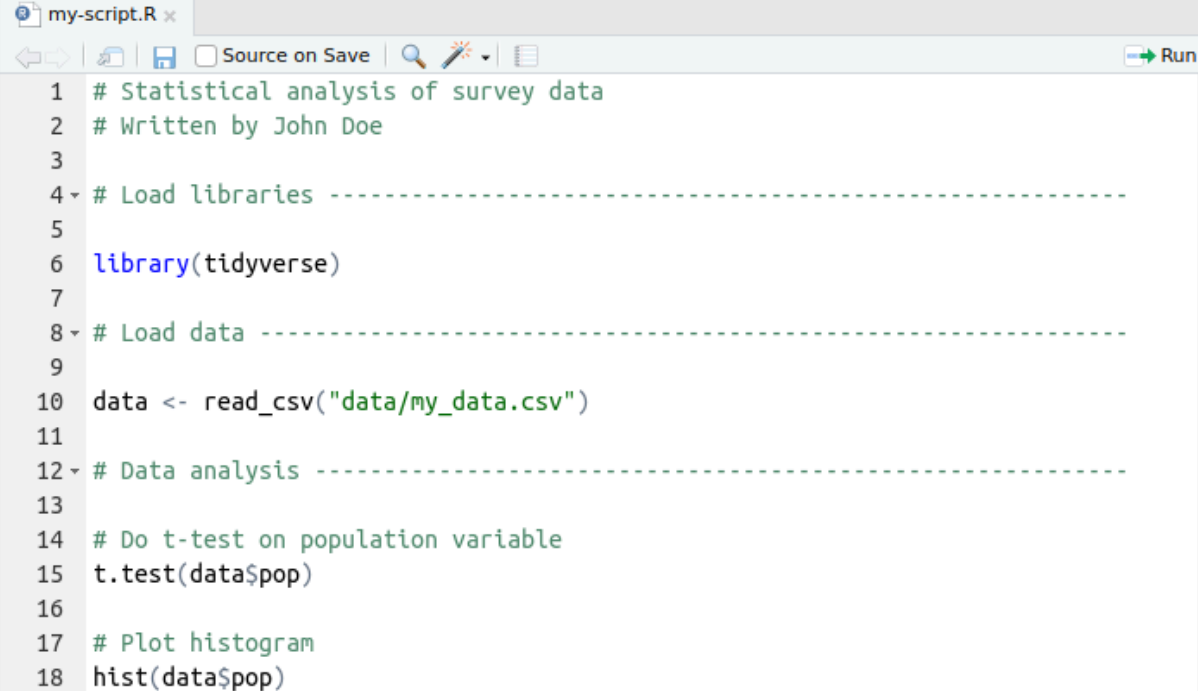
Review the previous section on [examples of best practices in programming](#):

- Organise code within each script.
- Use comments to mark off sections of code.
- Naming conventions for naming objects.
- Use relative paths to make sure the code runs on any computer.

- If you want to read more about good coding style, have a look at the [tidyverse style guide](#).

Good versus bad coding practice

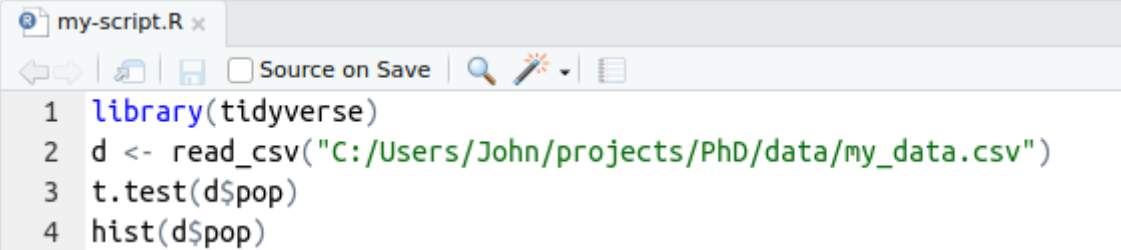
😊 Good coding



```
my-script.R x
Source on Save
Run

1 # Statistical analysis of survey data
2 # Written by John Doe
3
4 # Load libraries -----
5
6 library(tidyverse)
7
8 # Load data -----
9
10 data <- read_csv("data/my_data.csv")
11
12 # Data analysis -----
13
14 # Do t-test on population variable
15 t.test(data$pop)
16
17 # Plot histogram
18 hist(data$pop)
```

😞 Bad coding



```
my-script.R x
Source on Save

1 library(tidyverse)
2 d <- read_csv("C:/Users/John/projects/PhD/data/my_data.csv")
3 t.test(d$pop)
4 hist(d$pop)
```

Recognise good naming conventions

Naming conventions

Naming objects

There's no single correct way to name objects, but the following guidelines are recommended:

- Use only lowercase letters and numbers.
- Use names that are concise and meaningful (this is not easy!).
- Where possible, avoid re-using names of common functions and variables. This will cause confusion for the readers of your code.
- If you find yourself attempting to cram data into variable names (e.g. `model_2018`, `model_2019`, `model_2020`), consider using a list or data frame instead.

Examples:

- alllowercase: `adjustcolor`
- period.separated: `plot.new`
- underscore_separated: `numeric_version`
- lowerCamelCase: `addTaskCallback`
- UpperCamelCase: `SignatureMethod`

💡 Keep in mind, that using all lowercase letters might be difficult to read and that the period (.) and underscore (_) are used in many function names. Camel case is readable, but requires you to use the Shift button excessively. Choose one and use it consistently.

Naming functions

According to the [style guide by Hadley Wickham](#), variable and function names should be lowercase and you should use an underscore (_) to separate words within a name. Generally, function names should be verbs. As with objects, strive for names that are concise and meaningful (this is not easy!).

Where possible, avoid using names of existing functions and variables. Doing so will cause confusion for the readers of your code.

Naming output files

File names should end in `.R` and, of course, be meaningful.

Examples:

😞 `foo.R`

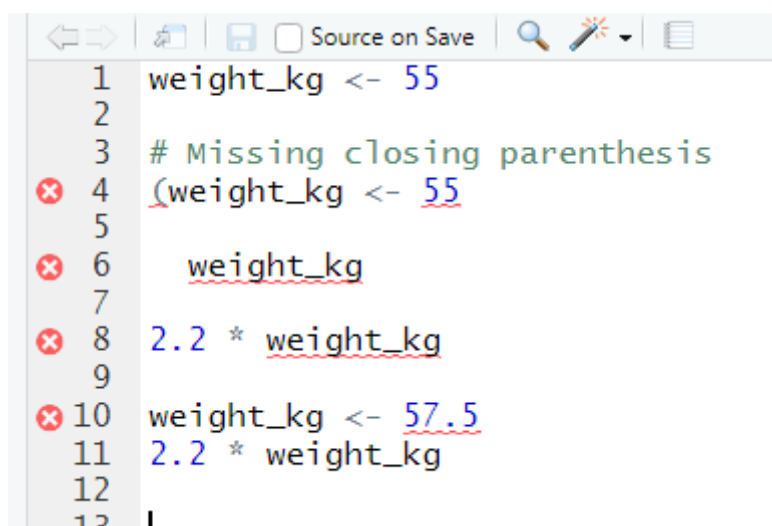
😊 `predict_ad_revenue.R`

📖 For more details, you can also have a look at [Google's R Style Guide](#).

Reading error messages

Errors in code

Don't get discouraged if your code doesn't run immediately! Error messages are common when programming, and fixing errors is part of any programmer's daily work. Often, the problem is a small typo in a variable name or a missing parenthesis. Watch for the red **x**'s next to your code in the **Script editor**. These may provide helpful hints about the source of the problem.



```
1 weight_kg <- 55
2
3 # Missing closing parenthesis
4 (weight_kg <- 55
5
6     weight_kg
7
8 2.2 * weight_kg
9
10 weight_kg <- 57.5
11 2.2 * weight_kg
12
13 |
```

If you can't fix an error yourself, start by googling it. Some error messages are too generic to diagnose a problem (e.g. "subscript out of bounds"). In that case it might help to include the name of the function or package you're using in your query.

Asking for help

If your Google search is unsuccessful, you may want to ask other R users for help. There are different places where you can ask for help. During this workshop, don't hesitate to talk to your classmates, compare your answers, and ask for help. You might also be interested in organising regular meetings following the workshop to keep learning from each other. If you have a friend or colleague with more experience than you, they might also be able and willing to help you.

Besides that, there are a few places on the internet that provide help:

- Join the ESCALATOR Slack workspace and ask questions there:
 - To join the workspace, click [here](#).
 - To go to the workspace if you have already joined, go to dhcssza.slack.com.
- [Stack Overflow](#): Many questions have already been answered, but the challenge is to use the right words in your search to find them. If your question hasn't been answered before and is well crafted, chances are you will get an answer in less than 5 min. Remember to follow their guidelines on [how to ask a good question](#).
- The [R-help mailing list](#): it is used by a lot of people (including most of the R core team). If your question is valid (read its [Posting Guide](#)), you are likely to get an answer very fast, but the tone can be pretty dry and it is not always very welcoming to new users.
- If your question is about a specific package rather than a base R function, see if there is a mailing list for the package. Usually it's included in the DESCRIPTION file of the package that can be accessed using `packageDescription("<package-name>")`.
- You can also try to contact the package author directly, by emailing them or opening an issue on the code repository (e.g., on GitHub).
- There are also some topic-specific mailing lists (GIS, phylogenetics, etc.). The complete list is [on the R mailing lists website](#).

Continue learning programming

Additional resources

The material we cover during this workshop will give you a taste of how you can use R to analyse data for your own research. However, to do advanced operations such as cleaning your dataset, using statistical methods, or creating beautiful graphics you will need to learn more.

The best way to become proficient and efficient at R, as with any other tool, is to use it to address your actual research questions. As a beginner, it can feel daunting to have to write a script from scratch, and given that many people make their code available online, modifying existing code to suit your purpose might get first hands-on experience

using R for your own work and help you become comfortable eventually creating your own scripts.

Workshops

If you feel that you learn best through instruction, then attending a hands-on [Carpentries workshop](#) is a good option.

Interactive learning

If you are interested in learning R in an interactive way, try the following:

- Explore the `{swirl}` package via the RStudio console:
 - Install the `{swirl}` package:

```
install.packages("swirl")
```

- Load the `{swirl}` package:

```
library(swirl)
```

- Follow the on-screen instructions to complete the tutorial.
- Work through the [RStudio Primers](#) online platform.

Videos, webinars and blogs

- If you prefer learning through instruction, have a look at the [free videos and webinars](#) on Posit's website.
- If you prefer to read, look at [The Posit blog](#).

RStudio education

If you're unsure where to start, choose your learning path from the [RStudio education site](#) to help you become productive in the minimum amount of time, based on your experience level.

Books

There are many [open source books](#) available. I would suggest that you start with the [R for Data Science](#) online (and free) book and move onto the more advanced topics from there.

A gentle introduction to tidy statistics in R

This [tutorial](#) covers the basics of research statistics in R, providing an emphasis on reading data into R, exploratory data analysis with the [Tidyverse](#), statistical testing with ANOVAs, and finally producing a publication-ready plot in [ggplot2](#).

Communities

Join the following communities or forums:

- ESCALATOR Workspace:
 - To join the workspace, click [here](#).
 - To go to the workspace if you have already joined, go to dhcssza.slack.com.
- Click on [R for Data Science Online Learning Community](#) to join their Slack workspace. You can ask questions about anything that is related to R.